

EnerGuard Starter Kit

Contenuto: `energuard_dataset.csv`, `train_baseline.py`, `oversight_manager.py`, `bias_detector.py`, `audit_logger.py`, `app.py`.

Avvio rapido:

```
pip install scikit-learn pandas streamlit shap joblib
python train_baseline.py          # genera modello.joblib e predizioni.csv
streamlit run app.py             # scheletro della dashboard
```

I TODO nei file indicano dove intervenire. Le sezioni obbligatorie della dashboard e i criteri con cui verrete misurati sono nel documento "Indicatori di qualità della dashboard": leggetelo PRIMA di scrivere codice.

Formati del dataset

- `energuard_dataset.csv`: formato standard (virgola, decimale punto), usato dal codice.
- `energuard_dataset_EXCEL.csv`: stessa tabella con separatore `;` e decimale `,`, si apre correttamente in Excel italiano.
- `energuard_dataset.xlsx`: versione Excel formattata con dizionario delle colonne. Il codice legge entrambi i CSV grazie a `utils_io.carica_csv()`.

Spiegazioni in linguaggio naturale (explainer.py)

`explainer.py` trasforma l'output del modello in una spiegazione comprensibile a un operatore di control room. Ha due motori intercambiabili:

- **Template** (default): deterministico, nessuna API, nessun costo. Funziona sempre.
- **LLM esterno**: chiamata via API a OpenAI, Anthropic, Azure OpenAI o qualunque endpoint compatibile (Ollama, vLLM, LM Studio). Si attiva creando un file `.env`.

Configurazione dell'LLM

1. Copiate `.env.example` in `.env` e compilate `LLM_PROVIDER`, `LLM_API_KEY`, `LLM_MODEL` (e `LLM_BASE_URL` per azure/compatible).
2. Verificate con `python test_llm.py`: stampa i fattori SHAP, la spiegazione a template e quella dell'LLM, con fonte e latenza.
3. `.env` non va mai messo nel repository o nella consegna: e' gia' in `.gitignore`.

Garanzie di progetto (rilevanti per la valutazione)

- **L'LLM non decide e non calcola**: riceve i numeri gia' prodotti dal modello e li traduce.
- **Guardrail**: se la risposta cita numeri assenti dai dati, usa gergo tecnico o non menziona il fattore principale, viene scartata e si usa il template.

- **Fallback automatico** su qualunque errore (rete, quota, timeout, JSON malformato): la dashboard non resta mai senza spiegazione.
- Ogni spiegazione dichiara la propria **fonte** (`template` , `llm:provider/modello` , `template(fallback:...)`), e la chiamata finisce nell'audit trail.

Cosa dovete fare voi

I TODO in `explainer.py` e le domande della guida riguardano: la scelta delle soglie in `ETICHETTE` , l'affinamento del prompt di sistema, e soprattutto la decisione se usare un LLM o il template e la relativa **giustificazione nella model card** (un LLM introduce non determinismo e dipendenza da un fornitore esterno: e' una scelta da motivare, non da dare per scontata).

EnerGuard · AI Human Oversight Hackathon · Deloitte × ENEL FNC · 2026